Assignment #1 – VinMotion

Locomotion and Mimic Policy Results
on the LimX HU_D03 Humanoid

NGUYEN HUYEN TRANG

+84 968 872 382 ◇ Ha Noi

Github ◇ htrang.cyb@gmail.com

26 October – 9 November 2026

Abstract

This report presents two policies trained for the **LimX Dynamics HU_D03** humanoid in **mjlab**: a **locomotion** policy that follows velocity commands on flat ground, and a **mimic** policy that imitates a dance clip. After 10,000 PPO iterations, the locomotion policy falls in only 1.3% of episodes and tracks linear velocity within about 0.27 m/s over a −1.5 to 2.0 m/s command range. The mimic policy reproduces a 15 s dance with a 360° turn: 99.2% of episodes run the full 10 s, and the tracked links stay 4.1 cm from the reference on average. All results are in simulation; neither policy has run on the real robot yet.

Keywords: humanoid locomotion, reinforcement learning, motion imitation, mjlab, PPO, LimX HU_D03

1 Overview

Both policies are built in the `hu-d03-rl` repository on top of mjlab. They reuse the Unitree G1 task designs and PPO hyperparameters, adapted to the HU_D03's joints, gains and links (Table 1).

Table 1: Summary of results (mean over the last 50 iterations).

	Locomotion	**Mimic**
Task	`Mjlab-Velocity-Flat-LimX-HU-D03`	`Mjlab-Tracking-Flat-LimX-HU-D03`
Run	`2026-10-28_05-49-07`	`2026-10-29_17-21-14`
Iterations	10,000	10,000
Episodes ending early	1.3%	0.8%
Main error	0.265 m/s (xy velocity)	4.1 cm (14-link position)
Hardware / time	1 Kaggle GPU, 5 h 56 min	1 Kaggle GPU, 7 h 51 min

2 HU_D03 model and shared setup

Both policies control the same model: a serial-joint HU_D03 with 31 joints (about 1.6 m tall), sending joint position targets at 50 Hz to PD controllers. Three things changed from the original plan:

- **Parallel linkages removed.** `hu_d03_spec.py` deletes the A/B linkages at the ankles and waist and their 6 `<connect>` constraints; the policy commands `ankle_pitch/roll` and `waist_roll/pitch` directly, as on the G1. The linkage mass (about 1.4 kg) is added to the parent bodies. On the real robot these targets must be converted to A/B motor commands.
- **Derived PD gains.** LimX does not publish its gains, so stiffness and damping are computed from each joint's armature with the G1 rule: 10 Hz natural frequency, damping ratio 2. At 5 Hz the robot could not hold its pose.
- **Flat 0.25 rad action scale.** The G1 rule `0.25 * effort / stiffness` gives only 0.03–0.05 rad per unit action on the legs, because the HU_D03's reflected inertias are large.

Table 2: Setup shared by both policies.

Item	Setting
Model	MJCF `HU_D03_03.xml`, 31 serial joints, 5 ms physics step
Effort limits	Legs 120 N·m; ankles and waist roll/pitch 90 N·m (two motors); arms 30 N·m; wrists, hands, head 18 N·m
Home pose	Base at 0.889 m; hips -0.25, knees 0.5, ankles -0.25 rad
Collisions	All geoms including self-collision; feet condim 3, friction 0.6
Networks	Actor and critic MLPs 512-256-128, ELU, observation normalisation
PPO	lr 10^{-3} adaptive (KL 0.01), $\gamma = 0.99$, $\lambda = 0.95$, clip 0.2, 5 epochs $\times$ 4 mini-batches, 24 steps per env
Compute	4096 parallel envs on a Kaggle T4, 10.5 h budget per run, resumes from the previous checkpoint

3 Locomotion policy

The locomotion policy (`Mjlab-Velocity-Flat-LimX-HU-D03`) learns to follow a velocity command (v_x, v_y, ω_z) on flat ground. A rough-terrain variant (`Mjlab-Velocity-Rough-LimX-HU-D03`) adds a terrain height scan and a terrain curriculum, but has not been trained yet(on-going).

3.1 Observations, rewards and curriculum

- **Actor observations:** base linear and angular velocity can be calculated in simulation from world coordination, in real-time run, they come from state estimation; projected gravity, joint positions and velocities, previous action, velocity command. The critic also gets privileged terms from foot contact, foot air-time, feet contact force from LimX Dynamics's robot model.
- **Episodes:** 20 s (1000 steps), ending early if the robot falls.
- **Curriculum by iteration:** $v_x \in [-1, 1]$ m/s, $\omega_z \in [-0.5, 0.5]$ rad/s until iteration 5000; then $v_x \in [-1.5, 2.0]$ m/s, $\omega_z \in [-0.7, 0.7]$ rad/s.
- **Posture:** loose tolerance on hip pitch and knees (0.3–0.35 rad when walking), tight on ankle roll (0.1) and waist roll/pitch (0.08–0.1).

Table 3: Locomotion reward terms on the HU_D03.

Term	Weight	Note
Linear velocity tracking	2.0	$\exp(-e^2/\sigma^2)$ on v_x, v_y
Angular velocity tracking	2.0	same for ω_z
Upright	1.0	body `waist_pitch_link`
Posture	1.0	near home pose, by standing/walking/running speed
Foot clearance	−2.0	
Joint position limits	−1.0	
Self-collision	−1.0	added for HU_D03, contacts > 10 N
Foot swing height	−0.25	
Action rate	−0.1	
Foot slip	−0.1	
Torso angular velocity	−0.05	added for HU_D03
Angular momentum	−0.02	added for HU_D03
Feet air time	0	disabled for HU_D03

3.2 Results

Run `2026-10-28_05-49-07` trained for 10,000 iterations (202 checkpoints) and was evaluated from its TensorBoard logs with `eval/evaluate.py`. Tracking quality is the mean exp-kernel reward per alive step (1 is perfect).

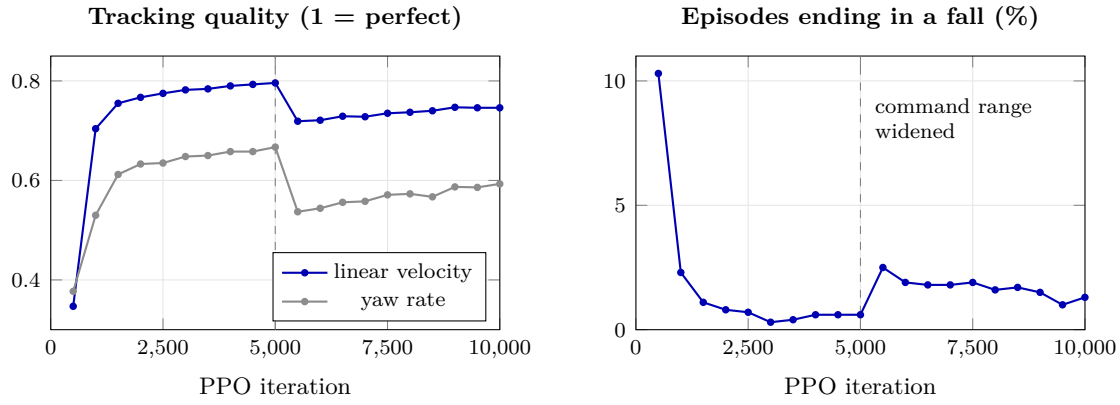


Figure 1: Locomotion policy: tracking quality and fall rate by iteration (mean of the 50 iterations before each point). Dashed line: command range widened at iteration 5000.

Table 4: Locomotion policy at selected checkpoints.

Checkpoint	v_x / ω_z range	Linear	Yaw	v_{xy} / ω_z error	Falls
1000	−1..1 / ±0.5	0.704	0.530	0.262 m/s / 0.406 rad/s	2.3%
5000	−1..1 / ±0.5	0.796	0.667	0.208 m/s / 0.304 rad/s	0.6%
9999 (final)	−1.5..2 / ±0.7	0.746	0.593	0.265 m/s / 0.360 rad/s	1.3%

The policy learns to stand and walk within about 1000 iterations and keeps improving until iteration 5000. When the command range widens, tracking quality drops by about 0.05 and has only partly recovered by iteration 10,000 (Figure 1). Yaw tracking stays weaker than linear tracking throughout.

4 Mimic policy

The mimic policy (`Mjlab-Tracking-Flat-LimX-HU-D03`) follows the BeyondMimic approach: one policy learns to reproduce a reference motion frame by frame.

4.1 Reference motion

The reference is the BONES-SEED clip `dance_basic_turn_v2_360_R_loop_001__A323`, a looping dance step with a full right turn, found by the notebook's search for "ballet". After conversion it has 749 frames at 50 fps (15.0 s) on the 31-joint model (Figure 2).

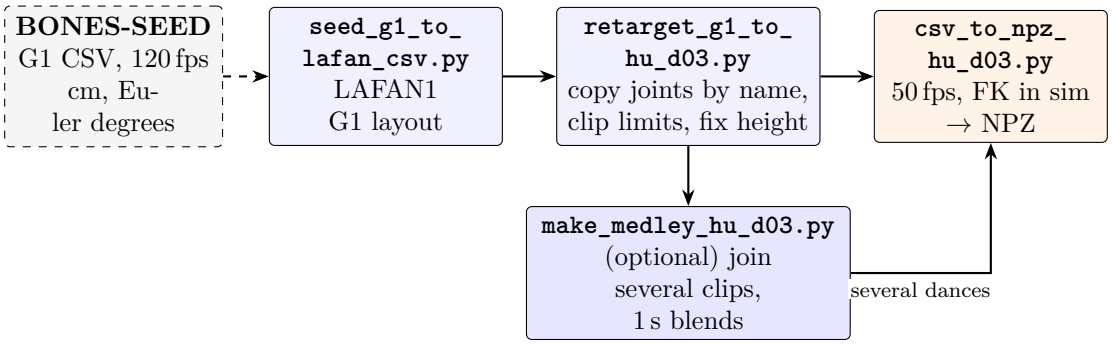


Figure 2: Preparing motion data for the HU_D03. Dashed boxes are external inputs.

The retargeting step copies joint angles from the G1 to the HU_D03 by name, choosing each sign by comparing joint axes at the zero pose. G1 wrist pitch/yaw map to HU_D03 wrist pitch and hand yaw; the elbows get a fixed offset; values are clipped to joint limits; root height is scaled by the pelvis-height ratio and shifted so the feet stay above the ground. The HU_D03 is about 0.3 m taller than the G1, so this copy keeps the style of the motion but only approximates foot and hand placement.

4.2 Observations, rewards and terminations

- **Actor observations:** reference joint targets, position and orientation error of the anchor (`waist_pitch_link`), IMU velocities, joint positions (with encoder bias) and velocities, previous action; noise-corrupted. The critic also gets the tracked bodies' poses.
- **Tracked bodies (14):** pelvis, hip roll, knee and ankle roll (both sides), torso, shoulder roll, elbow and wrist pitch (both sides).
- **Episodes:** 10 s (500 steps). An episode ends early if anchor height drifts > 0.25 m, anchor orientation error exceeds 0.8, or an ankle or wrist height drifts > 0.25 m.
- **RSI and adaptive sampling:** each reset places the robot at a clip frame; each second of the clip is one bin, and bins that fail more often are sampled more.
- **Robustness:** pushes every 1–3 s, randomised foot friction, torso centre of mass and encoder bias.
- **PPO:** same as locomotion but entropy coefficient 0.005, seed 42.

Table 5: Mimic reward terms.

Term	Weight
Anchor world position, exp kernel	0.5
Anchor world orientation	0.5
14 body positions, relative to anchor	1.0
14 body orientations	1.0
14 body linear velocities	1.0
14 body angular velocities	1.0
Action rate	−0.1
Joint limits	−10
Self-collision (> 10 N)	−10

4.3 Results

Run `2026-10-29_17-21-14` trained for 10,000 iterations (983 million env steps) in 7 h 51 min on one Kaggle GPU, at about 34,000 steps/s. Values come from `train.log`, averaged over the 50 iterations before each point.

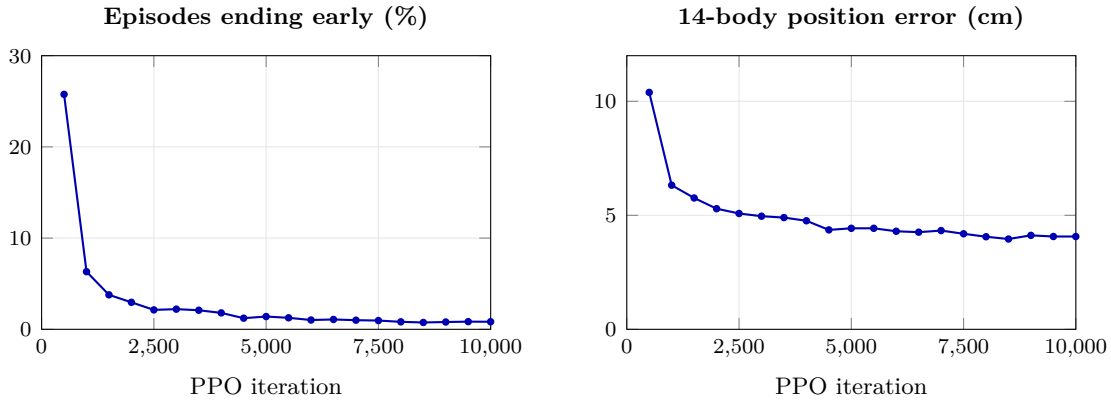


Figure 3: Mimic policy: share of episodes ending early and body position error by iteration, from iteration 500 (before that, nearly every episode ends early).

Table 6: Mimic policy at selected iterations.

Iteration	Length (/500)	Early end	Anchor (world)		14 bodies	
			position	orientation	position	orientation
500	427	26%	54.9 cm	0.169 rad	10.4 cm	0.398 rad
1000	485	6.3%	29.7 cm	0.110 rad	6.3 cm	0.224 rad
5000	496	1.4%	14.9 cm	0.076 rad	4.4 cm	0.125 rad
9999 (final)	498	0.8%	13.4 cm	0.071 rad	4.1 cm	0.113 rad

Most of the learning happens in the first 2000 iterations; after iteration 5000 both curves are nearly flat (Figure 3), so more training on this clip alone would add little.

4.4 Remaining weaknesses

- **Why episodes end early:** 99% of early endings are an ankle or wrist more than 0.25 m off the reference height; anchor-height endings are about 1% and anchor-orientation endings are zero. The robot almost never loses its balance.
- **Hardest part:** the most-sampled bin is about 73% through the clip (near 11 s), likely during the turn.
- **Anchor drift:** the links stay within 4.1 cm of the anchor, but the anchor itself drifts 13.4 cm in the world frame.
- **Joint angles drift from the reference:** joint-angle error fell to 0.97 at iteration 5000 and rose to 1.24 by the end, while body position error kept falling. The policy reaches the reference link positions with different joint angles, which fits a reference copied joint by joint from a shorter robot.

Table 7: Reward per term as a share of its maximum (last 50 iterations; per-second reward divided by weight, episodes nearly full length).

Term	Share of max
Body positions	97%
Anchor orientation	96%
Body orientations	90%
Body linear velocities	85%
Anchor position	80%
Body angular velocities	80%

4.5 Exported files

The run exported everything needed to replay or deploy the policy: the ONNX model (input: observations + `time_step`; output: action + reference frame), the `model_9999.pt` checkpoint, the reference NPZ, the `agent.yaml` and `env.yaml` configs, policy and reference videos, and `curves.png`.

4.6 Number of clips

The first policy learned one clip. The Kaggle notebook can join several clips (`MEDLEY`, currently six dances, about 112 s) into one continuous reference, and the code sets no limit on the number of clips. The practical limits are training time (10.5 h per run, and 15 s already took almost 8 h), the capacity of the 512-256-128 MLP, and the fact that the policy follows the time index, so it cannot choose a dance on command. With `WARM_START`, a few clips can be added per run, continuing from the previous checkpoint.

5 Comparing the two policies

Table 8: Locomotion and mimic policies on the HU_D03.

	Locomotion	**Mimic**
Goal	Follow a velocity command	Reproduce a reference motion
Command	Random (v_x, v_y, ω_z)	Time index into a clip
Generalises to	Any velocity in the trained range	Only the clip(s) it was trained on
Main reward	Velocity tracking (2.0 + 2.0)	Pose and velocity of 14 bodies
Style source	Hand-tuned posture and foot terms	The reference motion itself
Episode	20 s	10 s
Reset	Home pose	Random clip frame, hard parts more often
Curriculum	Command range widens at iteration 5000	Adaptive sampling
Entropy coefficient	0.01	0.005
Result	1.3% falls, 0.27 m/s error	0.8% early ends, 4.1 cm error

6 Limitations and next steps

1. **Locomotion:** train longer after the command range widens, or keep the iteration-5000 checkpoint for low speeds; improve yaw tracking (0.59 versus 0.75 linear); train and evaluate the rough-terrain task.
2. **Mimic:** train the `No-State-Estimation` variant, since the actor currently uses anchor position error and base linear velocity, which the real robot cannot measure reliably; re-evaluate without pushes or noise and check the error around the turn; train the six-dance medley from `model_9999.pt`.
3. **Retargeting:** retarget from human data with an IK tool (e.g. GMR) to fix foot and hand placement and the joint-angle drift.
4. **Sim-to-real:** replace the derived PD gains with LimX's real gains if available; convert ankle and waist targets to A/B motor commands before testing on the robot.

Sources

- `hu-d03-rl` repository: `src/hu_d03_mjlab/` (robot, tasks), `scripts/` (retargeting, data conversion), `notebooks/build_kaggle_*.py`.
- Locomotion: `eval/rows.json`, `eval/evaluate.py`, run `2026-10-28_05-49-07`.
- Mimic: `train.log`, `settings.json`, `params/agent.yaml`, run `2026-10-29_17-21-14`.
- mujocolab/mjlab, commit `8bb2bc9`: `tasks/velocity/velocity_env_cfg.py`, `tasks/tracking/tracking_env_cfg.py`.
- limxdynamics/humanoid-description: `HU_D03_description/xml/HU_D03_03.xml`.